

成长工具箱

Git 详细用法 · Markdown 写法 · KB 知识库教程

面向大一上科研与工程开发基础；排版：黑体标题、宋体正文、无装饰。

标注约定：〔已验证〕= 有公开方法论支撑；〔结构推定〕= 基于个人背景的判断，非普适事实。

一、Git 详细用法

Git 是版本控制系统。对你的价值不在「存代码」，而在四件事：时间机器（看任何文件历史演变）、实验沙箱（分支试错不污染主线）、备份防丢、可复现记录（commit 即日志）。

1.1 一次初始化

```
cd /path/to/study
git init
git config user.name "██████"
git config user.email "██████"
printf 'build/\n*.o\n__pycache__/\n.env\n.DS_Store\n' > .gitignore
git add .
git commit -m "init: ██████████"
```

先写 .gitignore 再 add，避免把编译产物、密钥提交。凭据 (.env) 不进版本库是硬约束。

1.2 每日循环（你真正每天敲的）

```
git status # ████████
git add kb/ # ███ kb █████ git add -A ████
git commit -m "kb: ███ ██████████status=███"
git push # ███ GitHub████████ push -u███ 1.6█
```

commit 粒度要小：一个知识点一提交，message 写清「哪篇 + 改了什么」。别攒一周才 commit，否则 diff 看不出演变。

1.3 查看与回退（时间机器）

```
git log --oneline # ████████
git log -p kb/cpp-pointer.md # ██████████→███
git diff HEAD~3 HEAD -- kb/ # ████████ kb
git checkout <commit> -- ███ # ████████████████
git revert <commit> # ████████████████████████
```

doc1 的「git 时间机器做差异对比」就是这里：复盘时直接 diff kb，不用自己记。

1.4 分支（实验沙箱）

```
git branch feat-handwriting # ██████████
git switch feat-handwriting # ████
# ... ████████████████
git switch main # ████
git branch -D feat-handwriting # ████████
# ██████████
git merge feat-handwriting
```

比「复制 backup1/backup2 文件夹」专业且省空间。日常用不到分支就别提前铺摊子（工具深度 > 广度）。

1.5 远程 GitHub

```
git branch -M main
git remote add origin git@github.com:████/████.git
git push -u origin main # ███ -u██████ git push
git pull --rebase origin main # █████/██████████████
```

推送的是你自己的 study 仓库（kb + 项目代码），不是 WorkBuddy 会话目录。

1.6 .gitignore 写法

```
# ████████
build/
*.o
*.out
# ████████
__pycache__/
# ██████████
.env
.DS_Store
```

规则：目录加斜杠，通配用 *，注释用 #。配一次即可。

1.7 删除文件

删 GitHub 上的文件本质是一次 commit。两种方式：

网页删：进仓库→点开文件→右上角垃圾桶→填说明→Commit（网页自动同步）。

本地删 + 推（推荐，和工作流一致）：

```
rm kb/■■■■■■.md          # ■ git rm kb/■■■■■■.md■■■ staged■
git add -A
git commit -m "remove: ■■■■■ kb/xxx"
git push
```

删的是最新版，历史里还在。若误传密钥，光删不够，需用 git filter-repo 清历史后 push --force（谨慎）。

1.8 常见坑

现象	原因 / 解法
push 被拒: non-fast-forward	建仓库时勾了 README，本地没有。先 git pull --rebase origin main 再 push。
HTTPS 总让输密码	旧方式已改 token 制。改用 SSH（配一次免密，见下）。
ssh: permission denied	公钥没加到 GitHub SSH keys；或 ssh-keygen 后没 cat 公钥去粘贴。
误提交密钥	.env 已进历史 → 用 git filter-repo 清，再 push --force；之后靠 .gitignore 防。

1.9 配置 SSH（一次性，macOS 免密）

```
ssh-keygen -t ed25519 -C "■■■■" # ■■■■■
cat ~/.ssh/id_ed25519.pub      # ■■■■■ GitHub→Settings→SSH keys
ssh -T git@github.com          # ■■ successfully authenticated ■■■
```

二、Markdown 写法

KB 和笔记都用 .md 纯文本写。语法少而够用，下面是可用的全集。

2.1 标题与段落

```
# 一级标题（黑体感）
## 二级标题
### 三级标题
```

这是一段正文。空一行表示分段。
换行用两个空格加回车，或直接空一行。

2.2 强调与列表

```
**加粗** *斜体* `行内代码`
```

- 无序项一
- 无序项二
- 1. 嵌套有序
- 2. 第二项

> 引用：写自己的话重构结论，不要抄定义。

2.3 代码块

行内代码用反引号：`git commit`。

```
```bash
git add -A && git commit -m "kb: ..."
```

```cpp
int a[5] = {1,2,3,4,5};
int* p = a; p = p + 1; // *p == 2
```
```

2.4 表格

| 层 | 角色 | 内容 |
|--------|-----|---------------|
| log | 过程层 | 今天做了什么、卡在哪 |
| kb | 沉淀层 | 用自己的话重构的知识 |
| review | 元层 | 规律 / 纠错 / 调方向 |

2.5 链接与图片

[链接文字](https://example.com)
![图片说明](path/to/img.png)

2.6 Frontmatter（KB 状态标记）

在 .md 文件最前面用 --- 包裹 YAML，记录元数据。状态标记贯穿 kb 全生命周期。

```
---
status: 新学 # 新学 → 会了 → 忘了（忘了就标回去）
date: 2026-10-01
tags: [cpp, 指针]
```

指针与数组退化

三、KB 知识库教程

3.1 三层记录法（禁止混写）

来自方法论 doc1。三层各司其职，写错层等于没记录。

| 层 | 角色 | 写什么 | 何时写 |
|--------|-----|------------------------|---------|
| log | 过程层 | 今天做了什么、卡在哪、踩了什么坑（流水账） | 每天结束 |
| kb | 沉淀层 | 把 log 里有价值的用自己话重构成知识 | 值得留存的当即 |
| review | 元层 | 规律发现 / 纠错 / 方向调整（不是流水） | 周 / 月固定 |

关键铁律：log 记过程、kb 记沉淀、review 记元层，三者分仓，禁止混写。

3.2 四段模板

每条 kb 用四段结构（具体四段以 doc1 源文档为准，以下为可直接用的结构〔结构推定〕）：

- 一、概念与原理：用自己的话讲清，不抄定义。
- 二、代码 / 示例：最小可运行例子（你的「内存布局 / 生命周期」坑就记在这）。
- 三、坑与边界：什么时候会错、UB、易混点（如 f1 悬垂指针、delete 后读）。
- 四、迁移与应用：能用在哪、能教谁（doc2 验收铁律：能教 = 真会）。

3.3 状态标记

每条 kb 头部 status 字段在 新学 → 会了 → 忘了 之间流转。复盘时改 md 头再 commit，进度一眼可见。忘了就标回「新学」补一次。

3.4 完整示例（一条 kb 长这样）

```
---
status: 新学
date: 2026-10-01
tags: [cpp, 指针, 数组]
---

# 指针与数组退化

## 一、概念与原理
数组名在大多数语境下会退化为指向首元素的指针。
所以 `int a[5]; int* p = a;` 中 p 指向 a[0]。
p+1 不是地址 +1 字节，而是跳一个 int（如 4 字节）。

## 二、代码 / 示例
```cpp
int a[5] = {1,2,3,4,5};
int* p = a;
p = p + 1; // *p == 2
```
但 `sizeof(a)` 仍是整个数组字节数（20），
而函数参数 `void f(int x[5])` 里 sizeof(x) 是指针大小（8）。

## 三、坑与边界
- 数组作函数参数退化为指针，函数内算不出数组长度，要额外传 size。
- 返回局部数组地址 = 悬垂指针（栈帧回收后不可读）。

## 四、迁移与应用
```

能解释为什么手写链表要用 new 在堆上分配节点，
而不是返回栈上局部节点地址。能教别人讲清 p+1 的跳法。

3.5 KB + Git + 复盘 闭环

每日：写完 / 改完 kb → git add kb/ → git commit（小粒度）。

每周（周末 2h）：git log -p kb/ 看演变；把「新学」升「会了」的改 status 再 commit；踩的坑确认进了第 3 段。

每月：对照能力→验收，看攒的是不是真能力；方向台账在三条线间调权重。

review 记元层发现（如「手写结构比刷 OJ 更暴露内存模型缺口」），不记流水。

附录 A：推荐目录结构

```
study/                                # 
kb/                                   # 
  cpp-pointer.md
  cpp-stack-heap.md
log/                                  # 
projects/                             # 
.gitignore
README.md
```

附录 B：最小命令速查

| 目的 | 命令 |
|---------|---|
| 初始化 | git init && git config user.name "x" && git config user.email "x" |
| 暂存并提交 | git add -A && git commit -m "类型: 说明" |
| 看历史 | git log --oneline |
| 看某文件演变 | git log -p 路径 |
| 对比差异 | git diff HEAD~3 HEAD -- 路径 |
| 关联远端 | git remote add origin git@github.com:u/repo.git |
| 首次推送 | git push -u origin main |
| 日常推送 | git push |
| 拉取并接后 | git pull --rebase origin main |
| 删文件 | git rm 路径 && git commit -m "remove: ..." |
| 开 / 切分支 | git branch x && git switch x |
| 回主线 | git switch main |

本教程基于本会话已确认的 Git / Markdown / KB 用法整理；具体四段模板以 doc1 源文档为准。